run. Additionally, we can specify the name of the package we want to clean.

- *yarn config set cache-folder <path>:* Sets *cache-folder* config value to configure the cache directory.

yarn clean: This command frees up space by removing unnecessary files and folders from package dependencies. It is useful in an environment where packages are checked into the version control directly.

On command execution, Yarn will create a *.yarnclean* file that should be added to version control. Cleaning is then automatically done as part of *yarn install* (or simply *yarn*) and *yarn add*.

> ***Note:*** As a best practice, it is recommended that you do not use this command. This command uses a heuristic to identify files that may not be needed from a distributed package and may not be entirely safe. This command is recommended only if you experience issues with the number of files that are installed as part of *node_modules*.

yarn info <package> [field]: This command will fetch information about a package and return it in a tree format. The package need not have been installed locally.

Example: yarn info express or yarn info express *express@1.15.0*.

Note that, by default, *yarn info* will not return the *readme* field (since it is often very long). To explicitly request that field, use *yarn info react readme.*

Yarn commands for managing package owners: Developers can write their own package and publish it either in a private or public registry. A package 'owner' in the registry is a user who has access to make changes to a package. A single package can have as many owners as you want.

Owners have permission to do the following tasks:

1. Publish new versions of the package
2. Add or remove other owners of the package
3. Change the metadata for a package

The following table lists a few *yarn owner* commands and their applications.

yarn owner ls <package>	Lists all of the owners of a *<package>*.
yarn owner add <user> <package>	Adds the *<user>* as an owner of the *<package>*. You must already be an owner of the *<package>* in order to run this command
yarn owner rm *<user> <package>*	Removes the *<user>* as an owner of the *<package>*. You must already be an owner of the *<package>* in order to run this command.

Commands for publishing a package to the npm registry: Once a package is published, you can never modify that specific version, so take care before publishing it.

The following table lists a few *yarn publish* commands and their applications.

yarn publish	Publishes the package defined by the *package.json* in the current directory.
yarn publish [folder]	Publishes the package contained in the specified folder. Project *package.json* should specify the package details.
yarn publish –access <public|restricted>	The *–access* flag controls whether the npm registry publishes this package as a public package, or is restricted.
Yarn publish –new-version <version>	Skips the prompt for the new version by using the value of *version* instead.

Command for running a defined package script in *package.json*: Define a *scripts* objects in your *package.json* file like the one I have defined in the code given below:

```
{
        "name": "my-package-name",
        "scripts": {
                "build": "babelsrc-dlib",
                "test": "test-code"
        }
}
```

Here, executing the command *yarn run test* on console will execute the script named *'test-code'* defined in your *package.json.*

Yarn is highly compatible with npm. Projects built using Yarn can still be installed via npm, and vice versa. I have been using it for a long time and till now have not found any problems with it. The Yarn project is backed by companies like Google and Facebook; so I believe it will be developed actively.

Yarn is not supposed to replace npm; rather, it provides an improved set of features. It uses the same *package.json* file and saves dependencies to *node_modules*.

In conclusion, both npm and Yarn are great dependency management tools, but I prefer to use the latter. END

Reference

https://yarnpkg.com

By: Manish Sharma

The author has a master's degree in computer applications, and is currently working as a technology architect at Infosys, Chandigarh. He can be reached at *cloudtechgig@gmail.com*.